

Extraction d'information

Image d'origine



2.b. Implémentation récursive

```
import imageio
import numpy as np
import matplotlib.pyplot as plt
import queue

## to ignore warning
import warnings
warnings.filterwarnings("ignore")#, category=DeprecationWarning)

def iterative_fill(image, starting_x, starting_y, threshold, replacement_color):
    # Initialize a queue for exploring neighboring pixels
    q = queue.Queue()

    # Check if starting pixel is within image bounds
    if starting_x < 0 or starting_y < 0 or starting_x >= image.shape[0] or starting_y >= image.shape[1]:
        return

    # Add starting pixel to the queue
    q.put((starting_x, starting_y))

    # Loop until the queue is empty
    while not q.empty():
        x, y = q.get()

        # Check if pixel is within image bounds and above threshold
        if x < 0 or y < 0 or x >= image.shape[0] or y >= image.shape[1] or image[x, y] <= threshold:
            continue

        # Check if pixel is already colored
        if image[x, y] == replacement_color:
            continue

        # Color the pixel
        image[x, y] = replacement_color

        # Add neighboring pixels to the queue
        q.put((x - 1, y)) # left
        q.put((x + 1, y)) # right
        q.put((x, y - 1)) # up
        q.put((x, y + 1)) # down

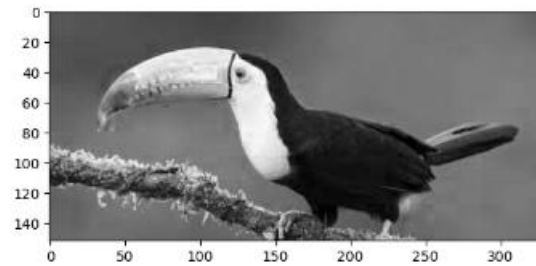
    # Example usage:
    img = imageio.imread('yoth.jpg')

    # Choose desired channel(s) for processing
    gray_image = img[:, :, 0] # grayscale
    # rgb_image = img # entire RGB image

    # Set configurable parameters
    threshold_value = 100
    replacement_color_value = 255

    # Call the iterative fill function from any starting pixel
    iterative_fill(gray_image, 10, 10, threshold_value, replacement_color_value)

    # Display the result
    plt.imshow(gray_image, cmap='gray')
    plt.show()
```



2.c. Implémentation avec une pile de voisins

```

import imageio
import numpy as np
import matplotlib.pyplot as plt
import sys

def iterative_fill(image, start_x, start_y, threshold, replacement_color):
    stack = [(start_x, start_y)]

    while stack:
        x, y = stack.pop()

        # Check if the pixel is within the image boundaries
        if x < 0 or y < 0 or x >= image.shape[0] or y >= image.shape[1]:
            continue

        # Check if the pixel value is above the threshold
        if image[x, y] > threshold:
            continue

        # Check if the pixel is already colored
        if image[x, y] == replacement_color:
            continue

        # Color the pixel
        image[x, y] = replacement_color

        # Add neighbors to the stack
        stack.append((x - 1, y)) # left
        stack.append((x + 1, y)) # right
        stack.append((x, y - 1)) # up
        stack.append((x, y + 1)) # down

    # Example usage:
    # Load the image and extract its layers
    img = imageio.imread('yoth.jpg')

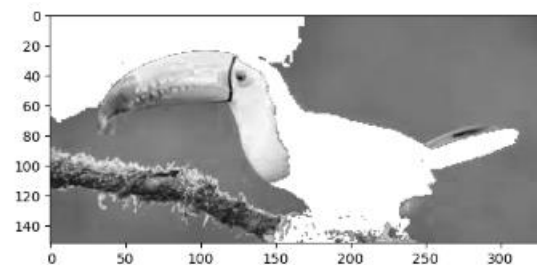
    # Assume you want to work with a single channel (grayscale image)
    gray_image = img[:, :, 0]

    # Set the threshold and replacement color
    threshold_value = 100
    replacement_color_value = 255

    # Call the iterative fill function for a specific pixel (e.g., pixel at (10, 10))
    iterative_fill(gray_image, 10, 10, threshold_value, replacement_color_value)

    # Display the result
    plt.imshow(gray_image, cmap='gray')
    plt.show()

```



3. Extraction de tâches

3.a. Algorithme et fonctions

```
from PIL import Image

def recursive_fill(image, threshold, x, y, target_color, replacement_color):
    """ Recursively fill a region in the image starting from the given pixel (x, y).
    Parameters:
    - image: PIL Image object
    - threshold: The threshold for coloring pixels
    - x, y: Coordinates of the starting pixel
    - target_color: Color to be replaced
    - replacement_color: Color to fill with
    - max_recursions: Maximum number of recursions allowed

    Returns:
    - Number of pixels colored in the region
    """
    # Get pixel value of the given coordinates
    pixel_value = image.getpixel((x, y))

    # Check if the pixel value is above the threshold and is of the target color
    if pixel_value > threshold and pixel_value == target_color:
        # Color the pixel with the replacement color
        image.putpixel((x, y), replacement_color)

        # Recursively fill the neighbors
        num_colored_pixels = 1 + \
            recursive_fill(image, threshold, x + 1, y, target_color, replacement_color, max_recursions - 1) + \
            recursive_fill(image, threshold, x - 1, y, target_color, replacement_color, max_recursions - 1) + \
            recursive_fill(image, threshold, x, y + 1, target_color, replacement_color, max_recursions - 1) + \
            recursive_fill(image, threshold, x, y - 1, target_color, replacement_color, max_recursions - 1)

    return num_colored_pixels
else:
    return 0

def analyze_image(image_path, threshold):
    """ Analyze the complete image, extracting tasks and calculating barycenters.
    Parameters:
    - image_path: Path to the image file
    - threshold: Threshold for considering pixels

    Returns:
    - List of barycenter positions with the number of pixels for each task
    """
    # Open the image
    image = Image.open(image_path).convert('L') # Convert to grayscale

    # Initialize variables to store barycenters and pixel counts
    barycenters = []
    pixel_counts = []

    # Iterate through each pixel in the image
    for x in range(image.width):
        for y in range(image.height):
            # Get pixel value at the given coordinates
            pixel_value = image.getpixel((x, y))

            # Check if the pixel value is above the threshold
            if pixel_value > threshold:
                # Initialize a new secondary image for visualization
                secondary_image = Image.new('L', (image.width, image.height), color=0)

                # Color the region starting from the current pixel and get the number of colored pixels
                num_colored_pixels = recursive_fill(image, threshold, x, y, target_color=255, replacement_color=128)

                # Calculate the barycenter
                if num_colored_pixels > 0:
                    cx = sum(i[0] for i in colored_pixels) // num_colored_pixels
                    cy = sum(i[1] for i in colored_pixels) // num_colored_pixels

                    # Append barycenter and pixel count to the lists
                    barycenters.append((cx, cy))
                    pixel_counts.append(num_colored_pixels)

    return barycenters, pixel_counts

# Example usage:
image_path = "yothe.jpg"
threshold = 128

barycenters, pixel_counts = analyze_image(image_path, threshold)

# Display the results
for i, (barycenter, pixel_count) in enumerate(zip(barycenters, pixel_counts)):
    print(f"Task {i + 1}: Barycenter {barycenter}, Number of Pixels {pixel_count}")
```

Travail demandé

1. Commencez par extraire la couche rouge

```
from PIL import Image

def extract_red_channel(image_path):
    image = Image.open(image_path)
    red_channel = image.split()[0] # Get the red channel
    return red_channel

# Exemple d'utilisation
image_path = "yothe.jpg"
red_channel = extract_red_channel(image_path)
```

2. On analyse l'image et on trace les positions des étoiles avec le nombre de pixels de la tache correspondante, qui donne une indication de la luminosité sur la couche rouge

```
: from PIL import Image

def extract_red_channel(image_path):
    image = Image.open(image_path)
    red_channel = image.split()[0] # Get the red channel
    return red_channel

# Exemple d'utilisation
image_path = "yoth.jpg"
red_channel = extract_red_channel(image_path)
```

```
threshold = 128
barycenters_red, pixel_counts_red = analyze_image(red_channel, threshold)

# Afficher les résultats
for i, (barycenter, pixel_count) in enumerate(zip(barycenters_red, pixel_counts_red)):
    print(f"Étoile {i + 1} (Rouge): Barycentre {barycenter}, Nombre de Pixels {pixel_count}")
```

3. Essayez de faire le même traitement sur la couche bleue

```
def extract_blue_channel(image_path):
    image = Image.open(image_path)
    blue_channel = image.split()[2] # Get the blue channel
    return blue_channel

# Exemple d'utilisation
blue_channel = extract_blue_channel(image_path)
barycenters_blue, pixel_counts_blue = analyze_image(blue_channel, threshold)

# Afficher les résultats
for i, (barycenter, pixel_count) in enumerate(zip(barycenters_blue, pixel_counts_blue)):
    print(f"Étoile {i + 1} (Bleue): Barycentre {barycenter}, Nombre de Pixels {pixel_count}")
```

4. La base de données Sloan Digital Sky Survey peut être interrogée directement par des requêtes SQL. L'utilitaire sqlcl.py permet de faire ces requêtes depuis un code python ou en ligne de commande.

```
: SELECT
    ra, dec, u_mag, g_mag, r_mag, i_mag
FROM
    YourSDSSTable
WHERE
    ra BETWEEN LowerLimit AND UpperLimit
    AND dec BETWEEN LowerLimit AND UpperLimit
```

5. Donnez la requête qui permet de récupérer des étoiles dans le champ stellaire identique à celui de l'image traitée plus haut. On récupère la position de ces étoiles et les magnitudes pour les couleurs U (ultraviolet), G (vert), R (rouge) et I (infrarouge proche). On trace les positions avec la valeur entière de la magnitude rouge

```
import matplotlib.pyplot as plt

# Exemple d'utilisation avec des données fictives
ra_data = [10, 15, 20, 25]
dec_data = [30, 35, 40, 45]
red_magnitude_data = [12, 15, 18, 20]

plt.scatter(ra_data, dec_data, c=red_magnitude_data, cmap='viridis')
plt.xlabel('RA (Right Ascension)')
plt.ylabel('Dec (Declination)')
plt.title('Positions des étoiles avec la magnitude rouge')
plt.colorbar(label='Magnitude Rouge')
plt.show()
```

